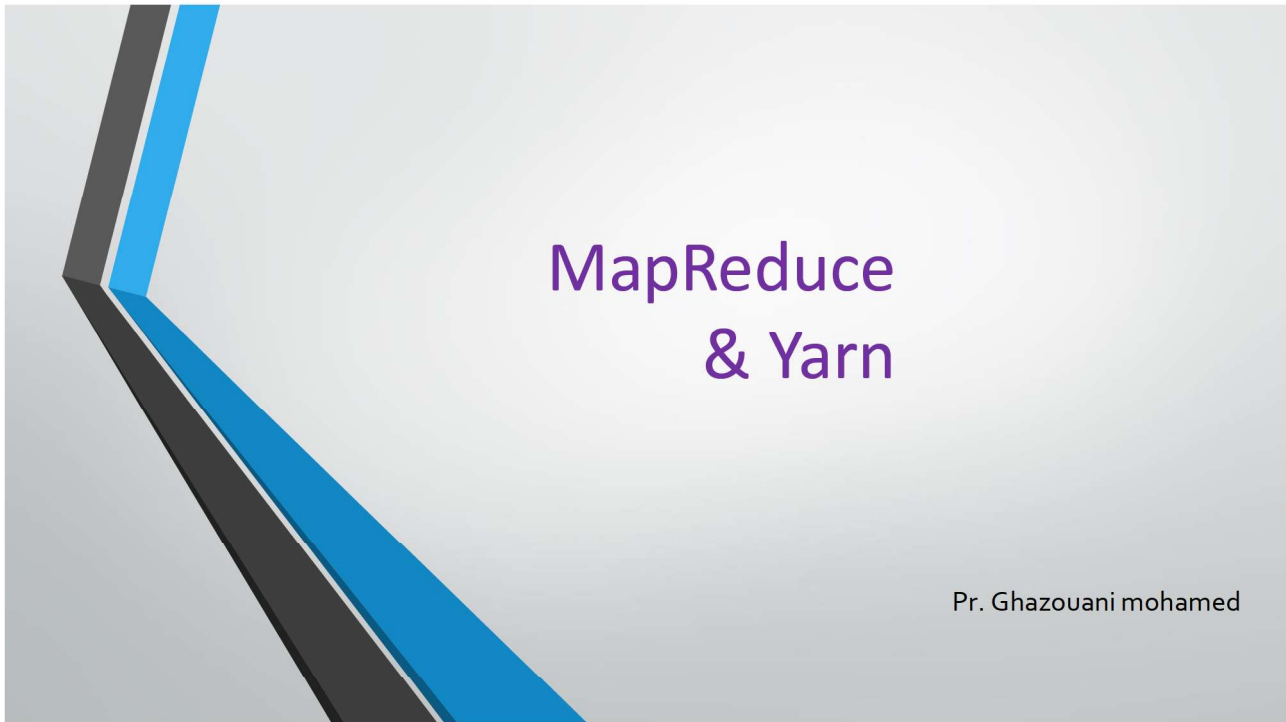
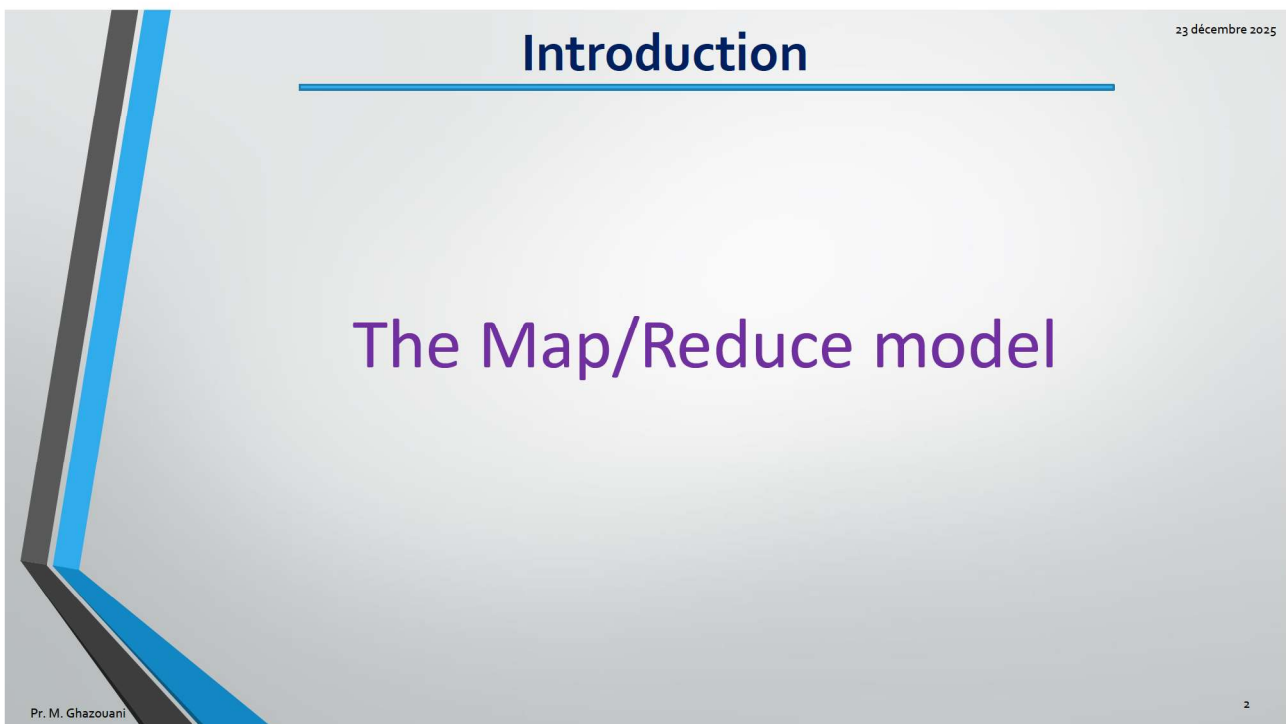




1



2

Introduction

23 décembre 2025

- To execute a large problem in a distributed manner, you must be able to break it down into several smaller problems that can be run on each machine in the cluster (an algorithmic strategy known as "**divide and conquer**").
- MapReduce is a paradigm (a model) that aims to generalize existing approaches to produce a single, unique approach **applicable to all problems**.
- MapReduce **had already existed** for a long time, particularly in functional languages (Lisp, Scheme), but the presentation of the paradigm in a "rigorous" form—one that is generalizable to all problems and oriented **toward distributed computing**—can be attributed to a white paper from Google's research department published in 2004 ("MapReduce: Simplified Data Processing on Large Clusters").

Pr. M. Ghazouani

3

3

Introduction

23 décembre 2025

MapReduce defines two distinct operations to be performed on input data:

- First, **Map**, which transforms the input data into a series of key/value pairs. It groups the data by associating them with keys chosen in such a way that the key/value pairs make sense in relation to the problem to be solved.
- Second, **Reduce**, which applies a process to all the values of each distinct key produced by the Map operation. At the end of the Reduce operation, a result will be obtained for each distinct key. Here, each machine in the cluster is assigned one of the unique keys produced by Map, along with the list of values associated with that key. Each machine then performs the Reduce operation for its assigned key.

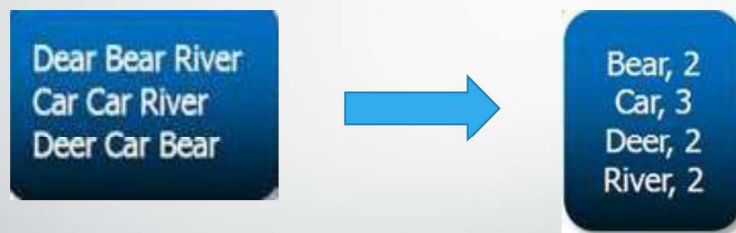
Pr. M. Ghazouani

4

4

Introduction

23 décembre 2025



Pr. M. Ghazouani

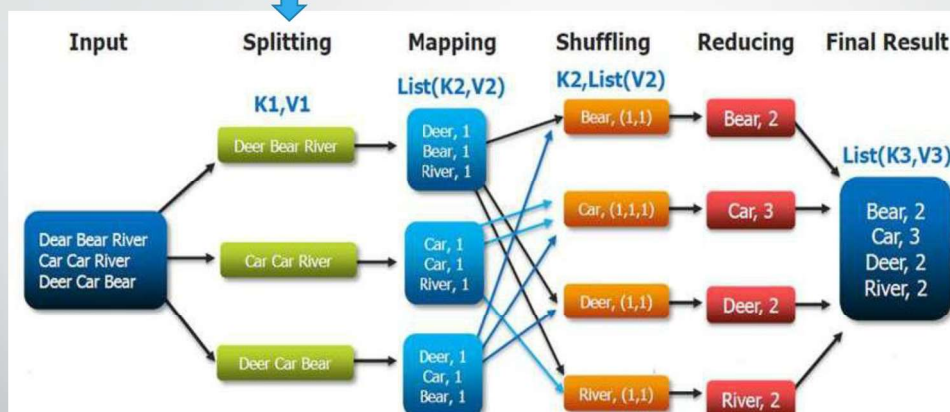
5

5

Introduction

23 décembre 2025

There are four distinct steps in a MapReduce process :



First, the document will be split into, for example, three data nodes. In each data node, there is a mapper or a map program (essentially a Java class, e.g., TokenizerMapper). Each mapper searches for key-value pairs. The key is the word, and the value is one.

Pr. M. Ghazouani

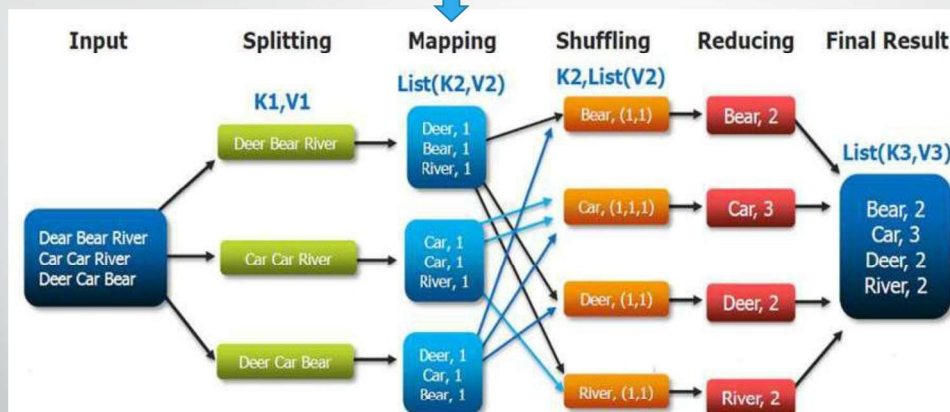
6

6

Introduction

23 décembre 2025

There are four distinct steps in a MapReduce process :



In the mapping phase, there's something called **tokenization**. This is where we use an analyzer to convert the content into a series of **tokens**, which in our case are words. We then need to specify the value we'll give to each token; in our case, that value is 1.

Pr. M. Ghazouani

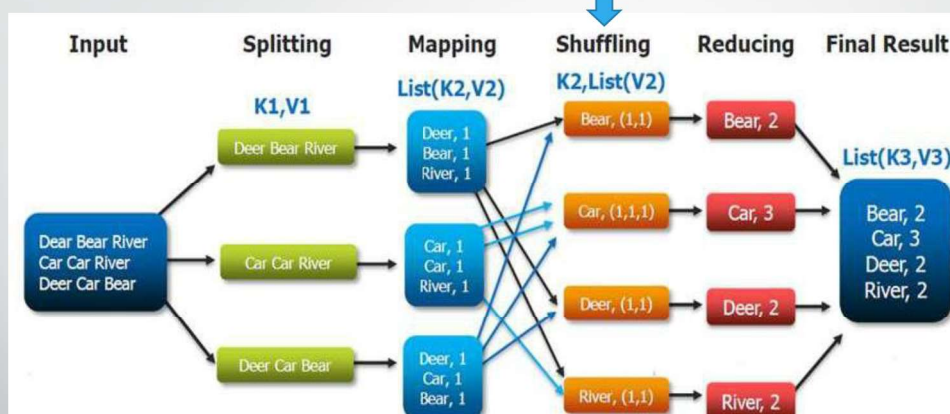
7

7

Introduction

23 décembre 2025

There are four distinct steps in a MapReduce process :



Afterward, a shuffle is applied, which is a **group by** and a **sort**. It keeps the word and counts how many times it appears. This is an internal process handled by The **JobTracker** (or **ResourceManager** in YARN), which we do not intervene with.

Pr. M. Ghazouani

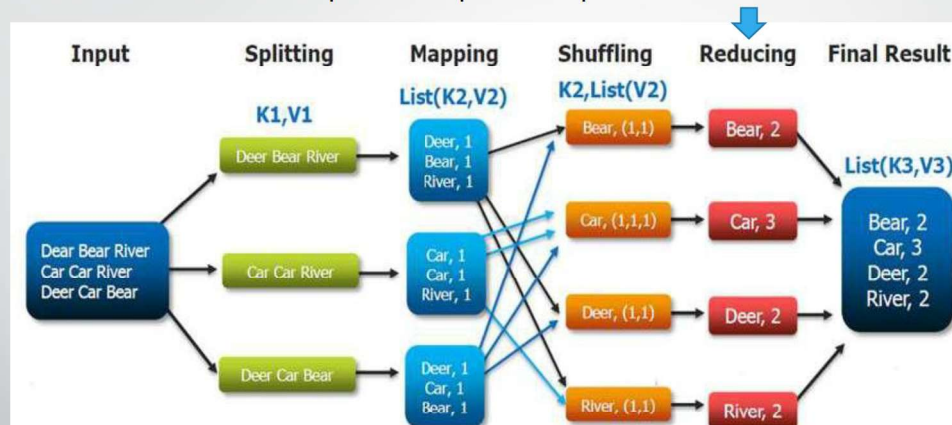
8

8

Introduction

23 décembre 2025

There are four distinct steps in a MapReduce process :



Finally, there's the **reducer program** (essentially a Java class, e.g., `IntSumReducer`), which will be executed and will sum the values for each key. The final result is a list of each word and the number of times it appears.

Pr. M. Ghazouani

9

9

Introduction

23 décembre 2025

To solve a problem using the MapReduce methodology with Hadoop, you'll need to:

- Choose a way to split the input data so the MAP operation can be parallelized.
- Define the KEY to be used for your problem.
- Write the program for the MAP operation.
- Write the program for the REDUCE operation.

...and Hadoop will handle the rest (distributed computing issues, grouping by distinct key between MAP and REDUCE, etc.).

Pr. M. Ghazouani

10

10

Concrete example

23 décembre 2025

Let's imagine we're given a text written in French. For a research project, we want to determine which words are most frequently used within this text.

Here, our input data consists of the content of the text.

First step: determine a way to **split** the input data so that each machine can work on a part of the text.

Our problem here is very simple—for example, we can decide to split the input data line by line. Each line of the text will be a fragment of our input data.

Pr. M. Ghazouani

11

11

Concrete example

23 décembre 2025

Our input data (the text):

He who believed in heaven
He who did not
[...]
Fool who plays the delicate one
Fool who dreams of his quarrels

"La rose et le Réséda" by
Louis Aragon, written in
1943.

To simplify things, we'll remove all punctuation and accented characters before the splitting. We'll also convert the entire text to lowercase.

Pr. M. Ghazouani

12

12

Concrete example

Our input data (the text):

- He who believed in heaven
- He who did not
- Fool who plays the delicate one
- Fool who dreams of his quarrels

... we get 4 fragments from our input data.

Pr. M. Ghazouani

23 décembre 2025

13

13

Exemple concret

We must now determine the key to use for our MAP operation and write the code for the MAP operation itself.

Since we're interested in the occurrences of words in the text, and we'll eventually have a result for each distinct key after the REDUCE operation, the logical key for our case is: the word itself.

As for our MAP operation, it will also be very simple: we'll just go through the fragment provided to us and, for each word, generate the key/value pair: (WORD, 1). The value here indicates the occurrence for this key—since we've encountered the word once, we give it the value '1'.

Pr. M. Ghazouani

23 décembre 2025

14

14

Concrete example

23 décembre 2025

The pseudo code for our MAP operation will therefore be:

For **WORD** in **LINE**, do:
Generate pair (**WORD**, 1)

For each of our fragments, the generated (key, value) pairs will be:

He who believed in heaven	→	(He;1) (who;1) (believed;1) (in;1) (heaven;1)
He who did not	→	(He;1) (who;1) (did;1) (not;1)
Fool who plays the delicate one	→	(Fool;1) (who;1) (plays;1) (the;1) (delicate;1) (one;1)
Fool who dreams of his quarrels	→	(Fool;1) (who;1) (dreams;1) (of;1) (his;1) (quarrels;1)

Pr. M. Ghazouani

15

15

Concrete example

23 décembre 2025

Once our MAP operation is completed (in a distributed manner), Hadoop will group (shuffle) all key-value pairs by their common key.

This operation is performed automatically by Hadoop. It is also carried out in a distributed way using a distributed sorting algorithm. After its execution, we will obtain the following 16 groups:

(he;1) (he;1)	(fool;1) (fool;1)	(his;1)	(the;1)
(who;1) (who;1) (who;1) (who;1)	(delicate;1)	(heaven;1)	
(believed;1)	(dreams;1)	(in;1)	(of;1)
(plays;1)	(did;1)	(not;1)	(one;1)
		(quarrels;1)	

Pr. M. Ghazouani

16

16

Concrete example

23 décembre 2025

We still need to create our **REDUCE** operation, which will be called for each distinct group/key.

In our case, it will simply consist of adding all the values linked to the specified key.

```
TOTAL = 0
FOR EACH ITEM in GROUP:
    TOTAL = TOTAL + 1
RETURN TOTAL
```

Pr. M. Ghazouani

17

17

Concrete example

23 décembre 2025

After the **REDUCE** operation is performed, a single value will be obtained for each distinct key. In this case, our result will be:

```
who : 4
he : 2
fool : 2
his : 2
the : 1
believed : 1
plays : 1
did : 1
one : 1
[...]
```

The most frequently used word in our text is "who," with 4 occurrences, followed by "he " and "fool," with 2 occurrences each.

Pr. M. Ghazouani

18

18

Concrete example: Web statistics

23 décembre 2025

Another example: we want to count the number of visitors on each page of a website. We have the log files in the following format:

```
/index.html [19/05/2017:18:45:03 +0200]
/contact.html [19/05/2017:18:46:15 +0200]
/news.php?id=5 [24/05/2017:18:13:02 +0200]
/news.php?id=4 [24/05/2017:18:13:12 +0200]
/news.php?id=18 [24/05/2017:18:14:31 +0200]
...etc...
```

What's our key?

Here, our key would be, for example, the page's access URL, and our **MAP** and **REDUCE** operations would be exactly the same as those just presented: this would give us the number of views for each distinct page on the site.

Pr. M. Ghazouani

19

19

Advantages of MapReduce

23 décembre 2025

- Scalability: We can add as many PCs as we want in parallel without stopping; once connected, the system configures them automatically.
- Cost-effectiveness: We can use ordinary, inexpensive PCs.
- Flexibility: We can program with Java, Python, R, Scala, etc. We can process text files, Excel, CSV, etc.
- Speed: The work is done in parallel on several machines at the same time.
- High availability: Thanks to fault tolerance.

Pr. M. Ghazouani

20

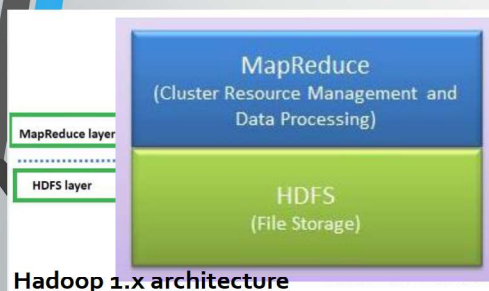
20

Hadoop architecture

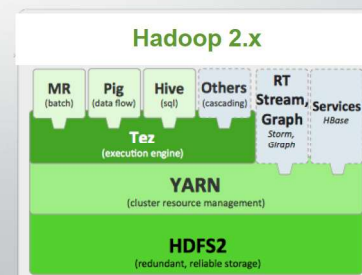
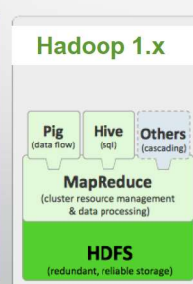
23 décembre 2025

There are two types of architectures:

- The first, consisting of the JobTracker and TaskTracker, corresponds to the first version of the MapReduce execution engine ("MRv1", Hadoop 1.x) and is presented for informational and historical reasons.
- Starting with version 2.x, Hadoop integrates a new execution engine: YARN ("MRv2"); this execution architecture, the current one, is also presented in a second part.



Pr. M. Ghazouani



21

21

MRv1

23 décembre 2025

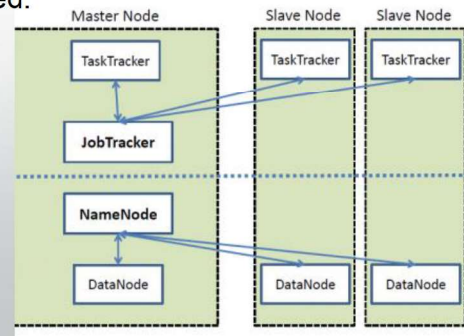
Hadoop's task management, up until version 2, was based on two services (daemons):

- The JobTracker

The JobTracker directly **receives the task** to be executed (a Java .jar file), along with **the input data** (the names of files stored on HDFS) and **the directory to store** the output data (also on HDFS). **There's only one JobTracker** on a single machine within the Hadoop cluster. The JobTracker communicates with the HDFS NameNode, so it knows where the data is located.

- The TaskTracker

The TaskTracker is in constant communication with the JobTracker and receives the simple operations to perform (MAP/REDUCE), as well as the corresponding data blocks (stored on HDFS). There's one TaskTracker on each machine in the cluster.



Pr. M. Ghazouani

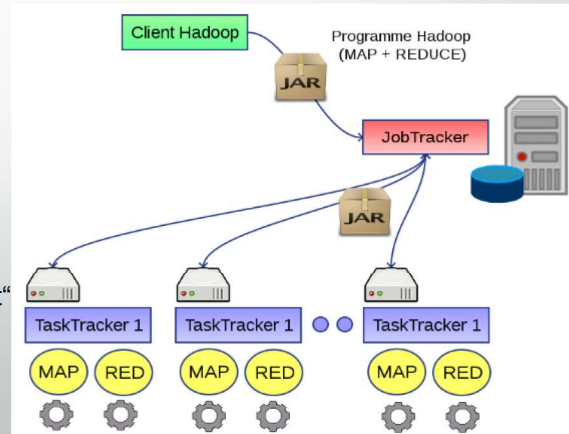
22

MRv1-The JobTracker

23 décembre 2025

An execution of a Hadoop task follows these steps from the JobTracker's perspective:

1. The client (a Hadoop console tool) submits the job to be performed to the JobTracker, including a **Java .jar** archive that implements the Map and Reduce operations. It also submits the **names of the input files** and the **location to store the results**.
2. The JobTracker communicates with the HDFS NameNode to find out where the data blocks corresponding to the file names given by the client are located.
1. From this information, the JobTracker determines which TaskTracker nodes (those that contain the data) will be assigned the task.
1. TaskTrackers regularly send a "heartbeat" to the JobTracker.



Pr. M. Ghazouani

23

YARN (MapReduce 2)

23 décembre 2025

YARN (Yet-Another-Resource-Negotiator) is also known as MRv2 (MapReduce 2). It's not a **complete redesign** but rather an evolution of the MapReduce framework.

- This new execution engine: **YARN manages cluster resources (such as disk, memory, and CPU)** for the various applications running on Hadoop. It also allows for the **management and scheduling of tasks** to ensure **optimal resource utilization**.
- When multiple applications are running on a Hadoop cluster, YARN makes sure each **application has enough resources** by allocating them in a balanced way.

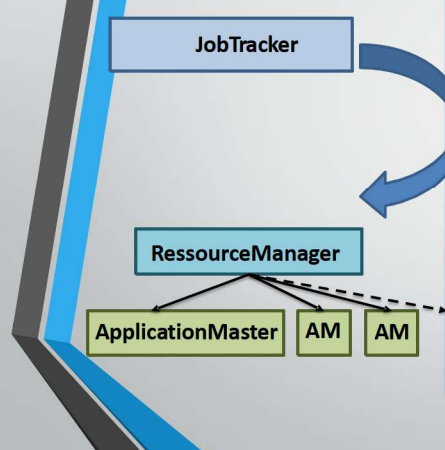
Pr. M. Ghazouani

24

24

YARN (MapReduce 2)

23 décembre 2025



The JobTracker has too many responsibilities:

- It manages cluster resources.
- It manages all jobs:
 - It allocates and schedules tasks.
 - It monitors task execution.
 - It handles failover.

Rethink the JobTracker's architecture.

- Separate cluster resource management from job coordination.
- Use slave nodes to manage jobs.

ResourceManager and ApplicationMaster.

- The ResourceManager replaces the JobTracker and only manages cluster resources.
- An ApplicationMaster entity is allocated per application to manage the tasks.
- The ApplicationMaster is deployed on the slave nodes.

Pr. M. Ghazouani

25

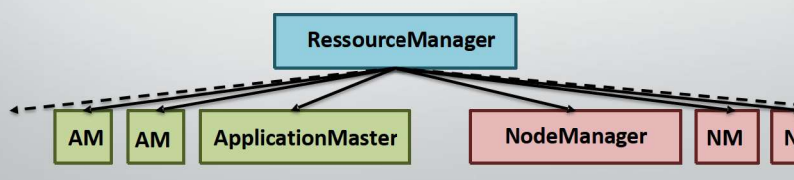
YARN (MapReduce 2)

23 décembre 2025

This new version also contains another component:

The NodeManager (NM)

The NodeManager is responsible for the resources of its own node and makes a portion of them available to the cluster.



Pr. M. Ghazouani

26

YARN (MapReduce 2)

23 décembre 2025

The JobTracker is gone from the architecture, or more accurately, its roles have been distributed differently.

Key Architectural Changes

- The architecture is now organized around a ResourceManager, which has a global scope for the entire cluster, and local ApplicationMasters, whose scope is limited to a single job or a group of jobs.
- In terms of responsibilities, you can say that:
JobTracker = ResourceManager + ApplicationMaster.
- A ResourceManager manages 'n' ApplicationMasters, and each ApplicationMaster manages 'n' jobs.

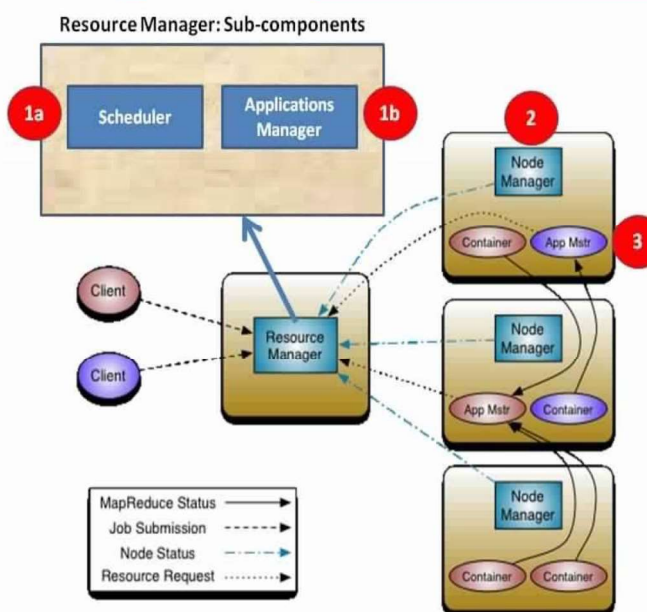
Pr. M. Ghazouani

27

27

YARN (MapReduce 2)

23 décembre 2025



ResourceManager

The **ResourceManager** is the client-facing replacement for the **JobTracker** in a Hadoop cluster. Its role has been streamlined to two main, distinct tasks:

- 1a) Scheduler (ResourceScheduler)**
- 1b) ApplicationsManager**

1a) Scheduler

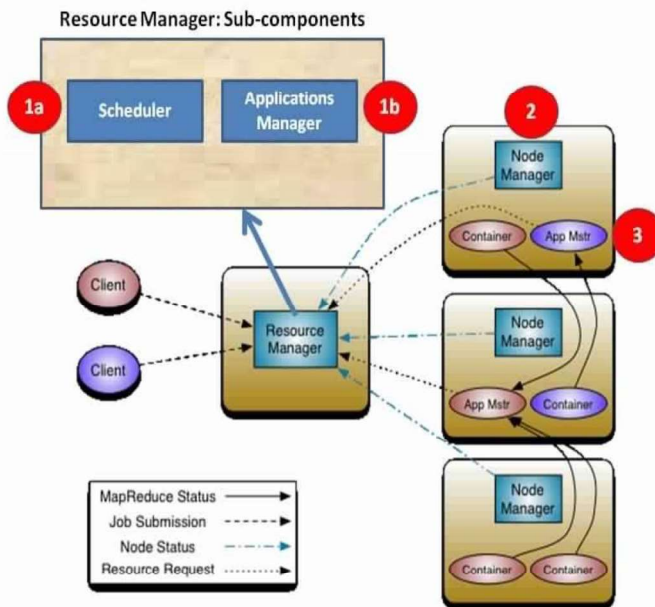
- The **Scheduler** determines how the available cluster resources are shared.
- It's responsible for allocating resources to applications running on the cluster.
- The resources allocated to these applications to allow them to run are called **Containers**.
- A **Container** is a bundle of resources, including memory, CPU, and disk space.

28

28

YARN (MapReduce 2)

23 décembre 2025



1b)- ApplicationsManager

The ApplicationsManager is the authority that manages the cluster's ApplicationMasters. As such, it is through the ApplicationsManager that you can:

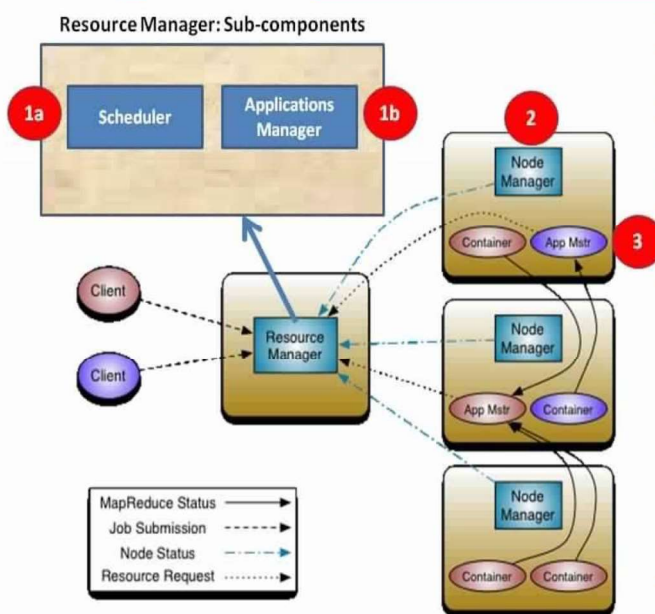
- Supervise the status of the ApplicationMasters
- Relaunch ApplicationMasters

29

29

YARN (MapReduce 2)

23 décembre 2025



2. NodeManager

A NodeManager runs on each node in the computer cluster. It receives commands from the **ResourceManager** and sends a periodic heartbeat signal. Each NodeManager is responsible for the resources of its own node and makes a portion of them available to the cluster as **Containers**.

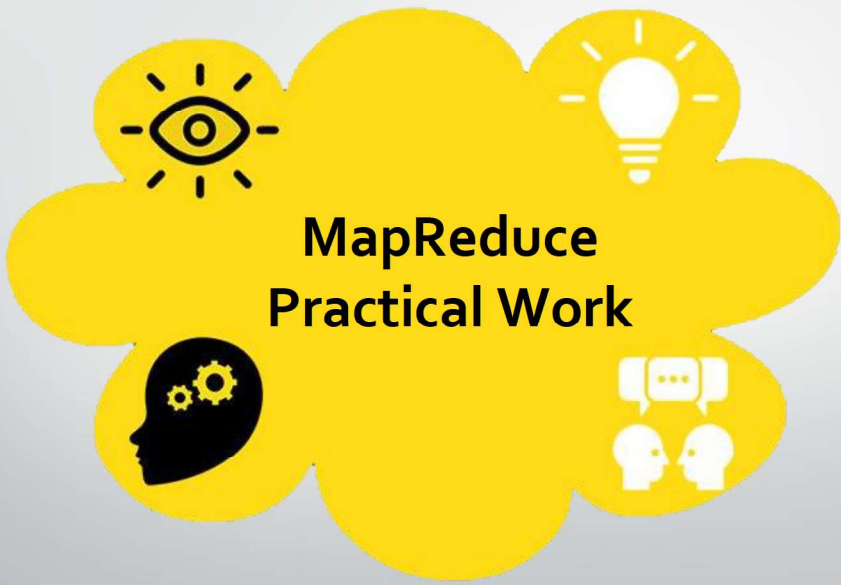
3. ApplicationMaster

- The ApplicationMaster is the component specific to each application, in charge of its associated jobs. It is responsible for:
 - Launching and, if necessary, relaunching jobs.
 - Negotiating for the necessary Containers with the Scheduler.
 - Monitoring the status and progress of jobs.
- From this perspective, an ApplicationMaster fulfills the role of a TaskTracker.

30

30

23 décembre 2025



The graphic features a large yellow cloud shape in the center. Inside the cloud, the text "MapReduce Practical Work" is written in black. Surrounding the text are four icons: a stylized eye with radiating lines (top left), a lightbulb (top right), a head profile with gears (bottom left), and two speech bubbles with three dots (bottom right).

Pr. M. Ghazouani

31

31

23 décembre 2025

Fin de la partie 5

Pr. M. Ghazouani

32

32